

Multi-Functional Robot Project Report

Team 10

**Student 1 Name: [Ahmed Aly Abbady]
Student 1 Code: [AA-037]**

**Student 2 Name: [Mohamed Mahmood atef]
Student 2 Code: [MM-017]**

1. Project Description

This project involves the design and development of a multi-functional robotic vehicle capable of operating in two primary modes.

Mode 1: Autonomous Line-Follower / Obstacle-Avoidance Car

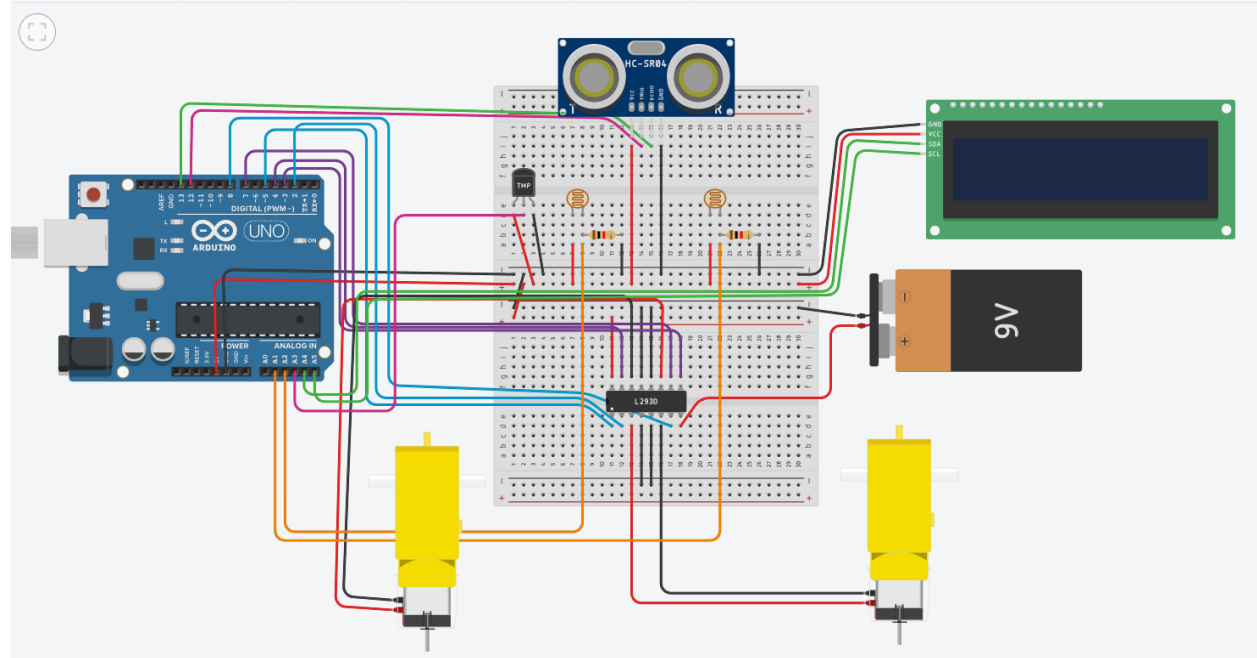
Mode 2: Bluetooth-Controlled Robotic Car

The robot is built around an Arduino microcontroller and integrates various sensors, including IR sensors for line detection, an ultrasonic sensor mounted on a servo motor for scanning and avoiding obstacles, and a Bluetooth module for remote control. Additionally, it features an environmental monitoring system using a DHT11 temperature sensor, with real-time data displayed on a 16x2 I2C LCD screen.

2. Components

- Arduino Uno (Microcontroller)
- L298N Motor Driver
- 4x DC Gear Motors with Wheels
- Ultrasonic Sensor
- Servo Motor
- 2x IR Sensors
- HC-06 Bluetooth Module (for manual control)
- DHT11 Temperature & Humidity Sensor
- 16x2 LCD Display with I2C Module
- Push Button (for mode switching)
- Arduino Sensor Shield (instead of standard breadboard)
- 2x 18650 Li-ion Batteries
- Robot Chassis, Battery Pack, mini breadboard, and Jumper Wires

3. Circuit Design



4. Working Principle

The robot operates using a state machine driven by a push-button toggle. Pressing the button switches the robot between 'Autonomous Mode' and 'Bluetooth Mode'.

- **Autonomous Mode:** The robot continuously reads the ultrasonic sensor. If an obstacle is detected within 15 cm, it moves backward slightly, and triggers the avoidance routine. The servo pans left and right to measure distances, and the robot turns towards the most unobstructed path, maneuvering until it regains its path. If no obstacle is present, it defaults to the line-following logic, using the two downward-facing IR sensors to track a black line.
- **Bluetooth Mode:** The robot disables autonomous behaviors and listens for serial commands via the Bluetooth module to move.
- **Environmental Display:** In parallel to its movement tasks, the robot checks the DHT11 sensor every 2 seconds and updates the LCD with the current temperature, alongside its distance readings and movement state.

5. Implementation

1. **Chassis Assembly:** The DC motors are mounted to the chassis. The L298N driver is connected to the motors.

2. Sensor Placement: IR sensors are placed at the bottom front facing downward. The servo is mounted on the top front with the ultrasonic sensor attached to it for a 180-degree sweep.
3. Control Hub: The Arduino is placed centrally. Power is routed from the batteries to the L298N, which then steps down power to the Arduino.
4. the LCD is mounted and A mini-breadboard is used to put the push button and the bluetooth module with the voltage divider
5. Wiring: Components are wired to the designated pins as mapped out in the code.
6. Programming: The code is compiled and uploaded via the Arduino IDE.

6. Code Explanation

Before the main code there are some important functions:

1. Core functions

```
128 void updateLCD() {
129     if (isAutoMode) {
130         lcd.setCursor(0, 0);
131         lcd.print("AUTO-M");
132     } else {
133         lcd.setCursor(0, 0);
134         lcd.print("B-MODE");
135     }
136 }
137
138 int getDistance() {
139     digitalWrite(trig, LOW);
140     delayMicroseconds(2);
141     digitalWrite(trig, HIGH);
142     delayMicroseconds(10);
143     digitalWrite(trig, LOW);
144     long duration = pulseIn(echo, HIGH, 30000);
145     int distance = duration * 0.034 / 2;
146     if (distance == 0) distance = 999;
147     return distance;
148 }
149
150 void runBluetoothMode() {
151     if (Serial.available() > 0) {
152         char cmd = Serial.read();
153         switch (cmd) {
154             case 'F': BmoveForward(); break;
155             case 'B': BmoveBackward(); break;
156             case 'L': BturnLeft(); break;
157             case 'R': BturnRight(); break;
158             case 'S': stopCar(); break;
159         }
160     }
161 }
```

2. Movement functions

```
165 void moveForward() {
166     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
167     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
168     analogWrite(ENA, speed); analogWrite(ENB, speed);
169 }
170
171 void moveBackward() {
172     digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
173     digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
174     analogWrite(ENA, speed); analogWrite(ENB, speed);
175 }
176
177 void turnRight() {
178     digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
179     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
180     analogWrite(ENA, rotation_speed); analogWrite(ENB, rotation_speed);
181 }
182
183
184 void turnLeft() {
185     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
186     digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
187     analogWrite(ENA, rotation_speed); analogWrite(ENB, rotation_speed);
188 }
189
190
191 void stopCar() {
192     digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
193     digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
194     analogWrite(ENA, 0); analogWrite(ENB, 0);
195 }
```

3. Bluetooth movement functions

```
196 void BmoveForward (){
197     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
198     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
199     analogWrite(ENA, 210); analogWrite(ENB, 210);
200 }
201 void BmoveBackward (){
202     digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
203     digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
204     analogWrite(ENA, 210); analogWrite(ENB, 210);
205 }
206 void BturnRight (){
207     digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
208     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
209     analogWrite(ENA, 230); analogWrite(ENB, 230);
210 }
211 void BturnLeft (){
212     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
213     digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
214     analogWrite(ENA, 230); analogWrite(ENB, 230);
215 }
```

4. Obstacle avoiding function

```

219 void avoidObstacle() {
220     stopCar();
221     lcd.setCursor(8, 0);
222     lcd.print("AVOIDING");
223
224     // 1. Scan Left and Right
225     my_servo.write(10);
226     delay(600);
227     int rightDist = getDistance();
228
229     my_servo.write(170);
230     delay(600);
231     int leftDist = getDistance();
232
233     my_servo.write(90);
234     delay(350);
235

```

```

236     if (rightDist > leftDist) {
237
238         turnRight(); delay(400);
239         stopCar(); delay(200);
240
241         moveForward(); delay(1200);
242         stopCar(); delay(200);
243
244         turnLeft(); delay(400);
245         stopCar(); delay(200);
246
247         moveForward(); delay(900);
248         stopCar(); delay(200);
249
250         turnLeft(); delay(300);
251         stopCar(); delay(200);
252
253         moveForward();
254         while(digitalRead(right_ir) == LOW && digitalRead(left_ir) == LOW) {
255             }
256         stopCar(); delay(100);
257

```

```

258     } else {
259
260         turnLeft(); delay(400);
261         stopCar(); delay(200);
262
263         moveForward(); delay(1200);
264         stopCar(); delay(200);
265
266         turnRight(); delay(400);
267         stopCar(); delay(200);
268
269         moveForward(); delay(900);
270         stopCar(); delay(200);
271
272         turnRight(); delay(300);
273         stopCar(); delay(200);
274
275         moveForward();
276         while(digitalRead(right_ir) == LOW && digitalRead(left_ir) == LOW) {
277             }
278         stopCar(); delay(100);
279     }
280
281     lcd.clear();
282     updateLCD();
283
284 }

```

➤ Obstacle avoiding logic

- The robot briefly reverses to create safe clearance, then the servo pans the ultrasonic sensor left and right to measure distances.
- The system compares both readings and directs the vehicle toward the path with greater clearance.
- The car maneuvers around the obstacle until the IR sensors detect the black line again to resume tracking.

Part 1: defining

```
1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <Servo.h>
4  #include <DHT.h>
5
6  #define echo 9
7  #define trig 10
8  #define right_ir 11
9  #define left_ir 12
10 #define ENA 5
11 #define IN1 7
12 #define IN2 2
13 #define IN3 4
14 #define IN4 8
15 #define ENB 6
16 #define button A1
17 #define DHTPIN A0
18 #define DHTTYPE DHT11
19
20 LiquidCrystal_I2C lcd(0x27, 16, 2);
21 Servo my_servo;
22 bool isAutoMode = true;
23 int lastButtonState = HIGH;
24 DHT dht(DHTPIN, DHTTYPE);
25 unsigned long lastTempUpdate = 0;
26 float currentTemp = 0.0;
27 int speed = 77;
28 int rotation_speed = 170;
```

1- Libraries inclusions (lines 1 to 4)

- Wire.h & LiquidCrystal_I2C.h: Enable I2C communication to control the 16x2 LCD display.
- Servo.h: Provides built-in functions for controlling the SG90 servo motor.
- DHT.h: Handles digital data communication with the DHT11 temperature and humidity sensor.

2- Pin definitions (lines 6 to 18)

3- Object instantiation & Global variables (lines 20 to 28)

- `lcd(...)`, `my_servo`, `dht(...)`: Instances initialized to interface with the LCD display, servo motor, and DHT sensor.
- `isAutoMode` & `lastButtonState`: Flag and state trackers for mode-switching logic (Autonomous vs. Bluetooth).
- `lastTempUpdate` & `currentTemp`: Non-blocking timing variables (`millis()`) to update temperature readings periodically without halting system execution.
- `speed (77)` & `rotation_speed (170)`: PWM motor speed parameters calibrated for straight motion and turning maneuvers.

Part 2: setup

```

30 void setup() {
31     my_servo.attach(3);
32     my_servo.write(90);
33     lcd.init();
34     lcd.backlight();
35     updateLCD();
36
37     pinMode(trig, OUTPUT);
38     pinMode(echo, INPUT);
39     pinMode(IN1, OUTPUT);
40     pinMode(IN2, OUTPUT);
41     pinMode(ENA, OUTPUT);
42     pinMode(IN3, OUTPUT);
43     pinMode(IN4, OUTPUT);
44     pinMode(ENB, OUTPUT);
45     pinMode(right_ir, INPUT);
46     pinMode(left_ir, INPUT);
47     pinMode(button, INPUT_PULLUP);
48
49     dht.begin();
50     Serial.begin(9600);
51 }

```

1- Actuator & Display initialization (lines 31 to 35)

- `my_servo.attach(3)` & `write (90)`: Binds the servo motor to digital pin 3 and centers it at 90° to point the ultrasonic sensor directly forward.
- `lcd.init()` & `lcd.backlight()`: Initializes the I2C LCD screen communication and enables its backlight.
- `updateLCD()`: Displays the initial operational mode on the screen upon boot.

2- Pin mode configuration (lines 37 to 47)

3- Peripherals initialization (lines 49 to 50)

- `dht.begin()`: Powers on and initializes digital data sampling from the DHT11 sensor.

- `Serial.begin(9600)`: Sets the serial communication baud rate to 9600 bps for receiving Bluetooth module commands.

Part 3: loop

1.3: Non-Blocking Temperature Monitoring (Lines 53–66)

```

53 void loop() {
54     // 1. Read Temperature every 2 seconds
55     if (millis() - lastTempUpdate >= 2000) {
56         lastTempUpdate = millis();
57         float t = dht.readTemperature();
58         if (!isnan(t)) {
59             currentTemp = t;
60
61             lcd.setCursor(9, 1);
62             lcd.print("T:");
63             lcd.print((int)currentTemp);
64             lcd.print("C");
65         }
66     }

```

1- Non-Blocking Timer (Line 55)

- Utilizes `millis()` subtraction (`millis() - lastTempUpdate >= 2000`) to execute the reading every 2 seconds without using `delay()`, ensuring vehicle motion and sensor tracking are never frozen or interrupted.

2- sensor reading & error handling

- Updates `lastTempUpdate` timestamp upon triggering.
- Reads the ambient temperature via `dht.readTemperature()`.
- Checks `isnan(t)` ("is Not-a-Number") to discard faulty sensor reads and avoid printing corrupt data.

3- LCD Display Update (Lines 61–64)

- Positions the LCD cursor at column 9, row 1 (`lcd.setCursor(9, 1)`).
- Casts `currentTemp` to an integer (`(int)currentTemp`) and displays the formatted temperature string (e.g., T:25C) on the right side of the screen's bottom row.

2.3

```
69   int buttonState = digitalRead(button);
70   if (buttonState == LOW && lastButtonState == HIGH) {
71       isAutoMode = !isAutoMode;
72       stopCar();
73       lcd.clear();
74       updateLCD();
75       delay(200);
76   }
77   lastButtonState = buttonState;
```

- Check button state to switch modes

3.3: Mode Check & Proximity Distance Display (Lines 79–92)

```

79 // 3. Execute Current Mode
80 if (isAutoMode) {
81     int distance = getDistance();
82
83     // Display Distance if too close
84     if (distance < 30) {
85         lcd.setCursor(0, 1);
86         lcd.print("D=");
87         lcd.print(distance);
88         lcd.print("cm ");
89     } else {
90         lcd.setCursor(0, 1);
91         lcd.print("    ");
92     }

```

1- Autonomous Mode Check (Line 80)

- Evaluates `if (isAutoMode)` to ensure the autonomous obstacle-monitoring routine runs only when the vehicle is set to Auto Mode.

2- Distance Acquisition (Line 81)

- Calls the modular function `getDistance()` to trigger the ultrasonic sensor and store the computed forward clearance in the local integer variable `distance`.

3- Proximity Warning on LCD (Lines 83–88)

- Evaluates `if (distance < 30)` to detect obstacles within a 30 cm alert threshold.
- Sets the LCD cursor to column 0, row 1 (`lcd.setCursor(0, 1)`).
- Prints the live formatted distance string (e.g., `D=25cm`) on the left side of the screen's bottom row.

4- Display Clearing (Lines 89–92)

- Executes the `(else)` block when no obstacle is detected within 30 cm.
- Overwrites the distance area with blank spaces (`lcd.print(" ")`) to clear old readings without overwriting the adjacent temperature display.

4.3: Autonomous Navigation Logic & Mode Branching (Lines 95–124)

```
95     if (distance > 0 && distance < 15) {
96         moveBackward();
97         delay(400);
98         avoidObstacle();
99     } else {
100         int right_ir_state = digitalRead(right_ir);
101         int left_ir_state = digitalRead(left_ir);
102
103         // Assuming Black line is centered (both sensors on White = LOW)
104         if (right_ir_state == LOW && left_ir_state == LOW) {
105             moveForward();
106             lcd.setCursor(9, 0); lcd.print("FWD   ");
107         } else if (right_ir_state == LOW && left_ir_state == HIGH) {
108             turnLeft();
109             lcd.setCursor(9, 0);
110             lcd.print("LEFT   ");
111         } else if (right_ir_state == HIGH && left_ir_state == LOW) {
112             turnRight();
113             lcd.setCursor(9, 0);
114             lcd.print("RIGHT  ");
115         } else if (right_ir_state == HIGH && left_ir_state == HIGH) {
116             stopCar();
117             lcd.setCursor(9, 0);
118             lcd.print("STOP   ");
119         }
120     }
121 } else {
122     runBluetoothMode();
123 }
124 }
```

1. Obstacle Trigger & Safety Maneuver (Lines 95–99)

- Evaluates if (distance > 0 && distance < 15) to detect critical frontal obstacles within 15 cm.
- Reverses the car via moveBackward() for 400 ms to create clearance before executing avoidObstacle().

2. IR Line Tracking Routine (Lines 99–120)

- Reads digital states from both line sensors via digitalRead(right_ir) and digitalRead(left_ir).
- **Straight Motion (Lines 104–106):** Drives forward (moveForward()) when both sensors detect the white surface (LOW/LOW), keeping the black line centered between them.

- **Left Turn Correction (Lines 107–110):** Triggers `turnLeft()` when the left sensor hits the black line (LOW/HIGH) to realign with the path.
 - **Right Turn Correction (Lines 111–114):** Triggers `turnRight()` when the right sensor hits the black line (HIGH/LOW) to restore orientation.
 - **Safety Stop (Lines 115–118):** Executes `stopCar()` when both sensors detect black (HIGH/HIGH), indicating an intersection, crossline, or end of track.
 - Updates the top-right corner of the LCD (`lcd.setCursor(9, 0)`) in real time to reflect the active maneuver (FWD, LEFT, RIGHT, STOP).
3. **Bluetooth Mode Fallback (Lines 121–123)**
- Executes `runBluetoothMode()` when `isAutoMode` is false, handing control over to manual serial commands.

7. Result

The integrated system successfully demonstrates seamless transitions between its operational modes. Upon powering on, the LCD accurately displays the current temperature. In autonomous mode, the vehicle effectively tracks the black line. When an obstacle is introduced, the ultrasonic sensor reads the distance and the Arduino triggers the avoidance sequence, allowing the robot to avoid the object and return to the line track. The manual override via Bluetooth functions as expected, allowing external control.

8. Cost Estimation

Component	Cost (EGP)
Arduino Uno	360
Chassis & DC Motors	400
L298N Motor Driver	75
Ultra sonic sensor	45
2x IR Sensors	60
HC-06 Bluetooth Module	200
DHT11 Sensor	75
16x2 I2C LCD	160
Batteries + Battery holder	110
Servo motor	75
Arduino Sensor Shield	75
Other components	40
Total	1,615